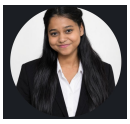
PROJECT AND TEAM INFORMATION

Project Title

Smart Water Management and Irrigation Optimization System Using C++

Student / Team Information

Team Name: *Team # DSCPP-III-2026-T294*	*aquacode*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Katiyar,Akanksha – 2510378334* *akankshakatiyar04@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Anand,Pragya – 2510011890* *anandpragya66@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Kumar Sahani,Shivendra – 2510370473* *shivendra7526048519sahani@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Agriculture needs appropriate and timely irrigation. However, traditional irrigation methods may fail to provide optimal water distribution for different areas within one farming sector, leading to over-irrigated plots and under-irrigated ones. In addition, water and electricity are precious resources, and they are wasted in the process. It is essential to develop a system that will be able to prioritize irrigation of the most needy areas. Smart Water Management and Irrigation Optimization System is the project I have chosen to simulate based on the described case. There are several factors that need to be considered in this study, including temperature, moisture, and the likelihood of rainfall. These factors will be used to determine which of the two given zones is in a higher need of irrigation. The zone with the lowest percentage of moisture, the highest temperature, and the least likelihood of rainfall will be prioritized. The primary purpose of my project is to demonstrate how the data structures and algorithms can be applied in a real-life scenario. The system will not control actual water pumps and sensors because it is a simulation, but in the future, it could be built into an actual system by adding the required hardware components.

State of the Art / Current solution (1 pt)

Irrigation systems today involve various levels of automation, ranging from manually operated ones and those controlled by timers to fully automatic ones utilizing sensors and controllers. Most basic autonomous systems are designed to simply turn the pump on when the level of moisture drops below a certain threshold. More advanced systems, however, also incorporate the ability to modify the irrigation based on the weather forecast, utilize multiple irrigation zones with individual controllers, and be remotely monitored or controlled. The project, apart from its primary concentration on automation, also demonstrates the use of several data structures. The sensor data is placed in a queue for further processing, while the priority queue selects the next irrigation zone to be watered. The entire system of water pipes can be represented as a graph or linked list, and the irrigation records can be stored in a linked list or vector. Finally, Dijkstra's algorithm is used to find the shortest path for water distribution in the system. Consequently, the project serves as an excellent illustration of DSA application in agriculture.

Project Goals and Milestones (2 pts)

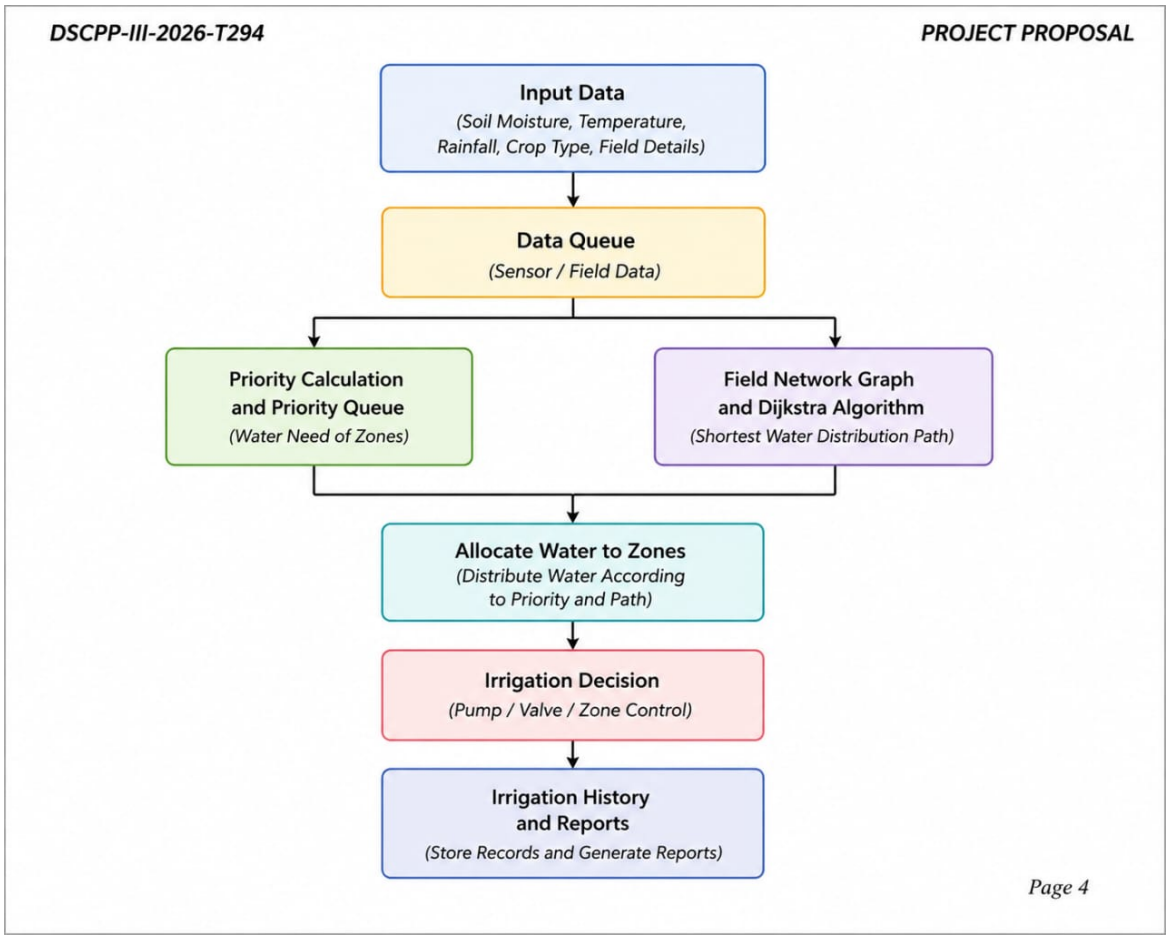
The main objective of the project is to develop a C++ application that can identify the irrigation needs of different farm zones and distribute limited water according to their priority. The major steps planned for the project are as follows: 1. Create a data model for farm zones, sensors, and irrigation details. 2. Use a queue to handle sensor readings as they arrive. 3. Calculate an irrigation priority using soil moisture, temperature, and rainfall probability. 4. Use a priority queue to select the zone with the highest irrigation requirement.

5. Represent the water-pipe network using a graph and adjacency list. 6. Apply Dijkstra's algorithm to find an efficient water-delivery route. 7. Store and maintain records of irrigation activities. 8. Develop a simple menu-driven interface and test the complete system. The final system will simulate sensor updates, decide the irrigation order, allocate available water, show the decisions made by the system, and keep a record of previous operations. The main objective of the project is to develop a C++ application that can identify the irrigation needs of different farm zones and distribute limited water according to their priority. The major steps planned for the project are as follows: 1. Create a data model for farm zones, sensors, and irrigation details. 2. Use a queue to handle sensor readings as they arrive. 3. Calculate an irrigation priority using soil moisture, temperature, and rainfall probability. 4. Use a priority queue to select the zone with the highest irrigation requirement. 5. Represent the water-pipe network using a graph and adjacency list. 6. Apply Dijkstra's algorithm to find an efficient water-delivery route. 7. Store and maintain records of irrigation activities. 8. Develop a simple menu-driven interface and test the complete system. The final system will simulate sensor updates, decide the irrigation order, allocate available water, show the decisions made by the system, and keep a record of previous operations.

Project Approach (3 pts)

The system will be developed in C++ using the C++ Standard Library. Each farm zone will hold data, including the ID, name of the crop, level of soil moisture, temperature, rainfall probability, and the amount of water required. The data from the sensors will be processed using a queue. The processing will generate the irrigation priority of each zone, which will be determined by having the lowest moisture level and highest environmental stress. The priority queue will pick the next most needed irrigation sector to perform the task. The water distribution system will be modeled using a graph. The data structure will use an adjacency list to hold the tank, pipes, and irrigation zones as nodes. The connections will have weights that will hold the distances or cost of transporting water through them. Dijkstra's algorithm will determine the shortest path in terms of distance or price. The system will also keep track of the irrigation records using a linked list or STL. The records can also be stored in a stack if the system is designed to provide an undo option for the manual entries. The project will be simple and use a menu-driven interface with sensors' simulated values. It will test the following scenarios: normal level of moisture, drought, a high probability of rainfall, low water levels, and a change in the pipe network.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The expected outcome is a working C++ console-based Smart Irrigation and Water Allocation System. The program will have the following features: Simulation of sensor readings for different farm zones Calculation of irrigation priority Priority-based allocation of limited water Graph-based representation of the water network Efficient route calculation using Dijkstra's algorithm Storage of irrigation history A menu-driven user interface During the final presentation, different sensor values will be provided for multiple zones. The program will process the provided values, determine which zones should be watered first, and find appropriate routes for the water to reach the targeted zones. The project will also demonstrate how the data structures such as queues, priority queues, graphs, adjacency lists, and others can be used in the real-life example

Assumptions

The project assumes that soil moisture, temperature, and rainfall probability values are available. Since this is a prototype, these values will either be simulated by the program or entered manually. The farm is divided into predefined irrigation zones, and the connections between these zones and the water source are already known. Water requirements are calculated using a simplified priority formula rather than a detailed agricultural model for every crop.
The weights assigned to the pipe connections represent distance, delivery cost, or another fixed measure used for route calculation. The prototype only simulates pump and valve operations and does not directly control physical irrigation equipment.

References

1.C++ reference documentation and the C++ Standard Library documentation.
2. Documentation for std::queue, std::priority$_q$ueue, std :: vectorandstd :: unordered$_m$ap.
3.Standard references on graph representation,adjacency lists, heaps and Dijkstra's shortestpathalgorithm.
4. FAO resources on efficient irrigation and agricultural water management.
5. Course notes and prescribed textbook for Data Structures using C++.